Tutorial Sheet 4

I/O and Interrupts

AUTHOR
Operating Systems Teaching and Research Unit

PUBLISHED
14.06.2024

Introduction

Question 1

In the lecture we discussed that the operation of I/O devices can be controlled by polling or via interrupts. However, in general, the operation through polling is not very efficient. In which cases could it still make sense to use polling? Why?

Question 2

Consider the following I/O devices on your computer:

- the mouse
- a hard disk with user files
- a light sensor on a smartphone

Decide for each device whether polling or interrupts are more suitable for I/O control and whether it makes sense to use an I/O buffer. Give reasons for your choice.

Question 3

Suppose a computer can read or write a memory word in 5 nanoseconds (nsec) Also suppose that when an interrupt occurs, all 32 CPU registers, plus the program counter (PC) and stack pointer (SP) are pushed onto the stack. What is the maximum number of interrupts per second this machine can process?

Question 4

In which of the four I/O software layers is each of the following done.

1. Computing the track, sector, and head for a disk read.
2. Writing commands to the device registers.
3. Checking to see if the user is permitted to use the device.
4. Converting binary integers to ASCII for printing.

Question 5

The clock interrupt handler on a certain computer requires 2 msec (including process switching overhead) per clock tick. The clock ticks at 60 Hz. What fraction of the CPU is devoted to the clock?

Question 6

A computer uses a programmable clock in square-wave mode. If a 500 MHz crystal is used, what should be the value of the holding register to achieve a clock resolution of

1. a millisecond (a clock tick once every millisecond)?
2. 100 microseconds?

Question 7

Assuming that it takes 2 nsec to copy a byte, how much time does it take to completely rewrite the screen of an 1920 × 1080 pixel graphics screen with 24-bit color? How many copies can we do per second? With a 60 Hz monitor, how many buffers do we need to avoid screen tearing?

Solution

Question 1

Polling can be more efficient than using interrupts in situations where I/O requests occur frequently and only have a short duration. If the request behavior is predictable, we can set up a polling loop accordingly and avoid the overhead of interrupt handling.

For instance, polling is ideal for a network bound program that frequently receives packets. In such cases, it's predictable that checking for new packets will usually result in work to process. The advantage is avoiding the synchronization and context switch overhead associated with interrupts. Moreover, the application can fetch data as needed, without being interrupted during ongoing operations.

Furthermore, polling could be useful if a device does not support interrupts or there are no more free interrupts available (and those already assigned cannot be shared).

Question 2

For the mouse, interrupt-driven I/O is most suitable, as mouse movements occur irregularly, but then need to be handled urgently. It also makes sense to use a buffer to hold its movement while higher priority operations take place.

For the hard disk, interrupt-based I/O control is also most suitable, as it is impossible to predict exactly when the data will be read or written. For efficient polling, we would need to know where the disk arm is at the moment, or how long the delay is when accessing the corresponding sector. Buffering is necessary to buffer data on the way from or to the disk (or even to buffer a block and calculate a checksum before passing it on to the operating system).

A phone's light sensor could use polling by fetching the data every second. This is not active polling, because the CPU is not reading the data as soon as possible. Rather, the OS will periodically read the sensor, interrupts would make no sense because we don't require low latency for the brightness of the phone screen to update to the light environment.

Question 3

We need (32 + 2) x 5 ns for storing the registers.

We need an additional (32 + 2) x 5 ns for getting the registers back at the end of the ISR.

The time for storing and restoring the register is 340 ns.

The maximum number of interrupts we can execute, if we neglect the ISR execution time, is 1 / 340e-9 ≈ 2,941,176, approximately 3 million interrupts a second.

Question 4

1. Device drivers or disk controller: This operation involves translating a logical block address into a physical location on the disk. Some disk controller are responsible for it it because the internal disk layout can be hidden to the OS, and if not, the device driver is responsible for that, because it understands the specifics of the hardware it is controlling.

2. Device drivers: Writing commands to the device registers is a low-level operation that involves direct interaction with the hardware. This is typically managed by the device driver.
3. Generic I/O layer: Access control and permission checking are higher-level functions that are not specific to any one device. This is managed by the operating system's kernel, which ensures that the user has the necessary permissions before allowing access to the device.
4. Device drivers: Converting data from the computer's internal binary format to a human-readable ASCII format is a typical function of this layer, as it standardizes data presentation regardless of the specific device involved.

Question 5

60 x 2 = 120 ms

120 / 1000 = 0.12

Therefore, 12% of the CPU time is devoted to the clock

Question 6

500 MHz represents a period of 2 nanoseconds (2 ns or 2e-9s)

For a millisecond (a clock tick once every millisecond):

1ms = 10e-3s

10e-3 / 2e-9 = 500000

For 100 microseconds:

100 µs = 100e-6

100e-6 / 2e-9 = 50000

In the first case, the register would need to be reset to 500000 after each interrupt, and 50000 for the second case.

Question 7

1920 x 1080 x 3 ≈ 6220800 bytes or 6 MiB frame buffer

With 2 ns per byte, it takes ~ 12 ms per copy, which means we can do ~80 copies per second.

To avoid screen tearing, we can use a second buffer:

- one buffer holds the frame that is currently being displayed
- at the same time, one buffer is being filled
- we swap the two buffers when the copy is done and while the screen is not refreshing

Tasks

Task 1

Disk requests come in to the disk driver for cylinders 10, 22, 20, 2, 40, 6, and 38, in that order. A seek takes 6 msec per cylinder. How much seek time is needed for

1. First-Come, First-Served (FCFS).
2. Shortest Seek First (SSF).
3. Elevator algorithm (SCAN, initially moving upward).

In all cases, the arm is initially at cylinder 20. There are 40 cylinders in total.

ⓘ Solution

1.

The requests are scheduled as they come. Total distance is calculated as follows:

- 20 -> 10 = 10
- 10 -> 22 = 12
- 22 -> 20 = 2
- 20 -> 2 = 18
- 2 -> 40 = 38
- 40 -> 6 = 34
- 6 -> 38 = 32

The total seek time is then multiplied by 6 and equals (10 + 12 + 2 + 18 + 38 + 34 + 32) x 6 = 876 ms

1.

The requests are now scheduled as follows:

20 -> 20 -> 22 -> 10 -> 6 -> 2 -> 38 -> 40

The total distance is (0 + 2 + 12 + 4 + 4 + 36 + 2) = 60

The total seek time equals 360. ms

1.

The requests are executed as follows:

20 -> 20 -> 22 -> 38 -> 40 -> 10 -> 6 -> 2

The total distance is (0 + 2 + 16 + 2 + 30 + 4 + 4) = 58

The total seek time equals 348 ms.

Conclusion: FCFS's seek time is 876 ms, SSF's seek time is 360 ms and SCAN's seek time is 348 ms.

Task 2

Assume a Network Interface Controller (NIC) has 5 different buffers. The index of the buffer currently in use is located in the controller register `nic_buffer_position_reg`.

Consider the following Interrupt Service Routine (ISR) code:

```
1  index = *nic_buffer_position_reg;
2  // buffer is a local array
3  copy_buffer_from_driver(&kbuf[index], &buffer);
4  schedule_upper_half(&buffer);
5  *nic_buffer_position_reg++;
6  ack_interrupt();
7  return_from_interrupt();
```

What I/O software design goal this interruption service routine violates? Think of a solution to fix it.

ⓘ Solution

This ISR violates the goal of ensuring data integrity and consistency. If another interrupt occurs during the execution of the ISR, especially before `nic_buffer_position_reg` has been incremented, the OS could end up copying the same network packet twice.

To fix this, a proper synchronization mechanism is needed. One simple solution is to disable (or mask) interrupts during the execution of this ISR to prevent any other interrupts from interfering:

```
1  disable_interrupts()
2  index = *nic_buffer_position_reg;
3  // buffer is a local array
4  buffer = copy_buffer_from_driver(&kbuf[index]);
5  schedule_upper_half(&buffer);
6  *nic_buffer_position_reg++;
7  enable_interrupts();
8  ack_interrupt();
9  return_from_interrupt();
```

This ensures that the ISR completes its critical section without interruption, maintaining data integrity.

Task 3

You are using a small computer board, such as a Raspberry Pi, to fetch the weather forecast and display it on a small LCD screen.

1. Which method would you choose to interact with the LCD controller: Programmed I/O, Interrupt Driven I/O, or DMA? Justify your choice.

Now, suppose you add a sensor to your computer. This sensor can read the room temperature and humidity directly from its controller's registers.

1. Which method is best suited if you want to read the values every 5 minutes?

2. Is there a better option if you want to read the values only when they change?

3. How would your approach change if the board were battery-powered instead of being plugged into a constant power source?

ⓘ Solution

1. If the only purpose of the device is to display the weather, Programmed I/O can be used effectively. While Interrupt Driven I/O and DMA are also viable, they may be unnecessarily complex for this simple task.

2. To read the values every 5 minutes, you can set a timer to read the sensor's registers at 5-minute intervals.

3. If you want to read the values only when they change, an Interrupt Driven I/O solution is better. This way, the sensor will notify the processor only when the data changes, avoiding unnecessary reads.

4. If the board is battery powered, avoid using Programmed I/O as it consumes more power due to constant polling. Instead, use Interrupt Driven I/O, which allows the CPU to remain idle most of the time, conserving battery life.

Task 4

A typical sound card is working at 48 kHz with each sample being stored on 16 bits. The sound card controller is using two buffers. While the first buffer is used for playing sounds on the sound card, the other one is being filled by the driver. The drivers is using DMA to copy audio data from the kernel to the audio controller.

Questions

1. What hardware buffer size should be used if we want to play 10ms audio data at a time?
2. What hardware buffer size should be used if we want to play 100ms audio data?
3. What is the difference in latency and CPU usage between 10ms and 100ms?
4. What is the advantage of using a double buffering technique?
5. How does Direct Memory Access (DMA) improve the efficiency of audio data transfer compared to CPU-driven data transfer?

Solution

1. The audio sample rate is 48,000 samples/second, with each sample being 2 bytes. The bandwidth for this audio card is 96,000 bytes/s (96 KB/s). For 10ms, it is then 96,000 / 100 ≈ 960 bytes, approximately 1 KB.

2. 96,000 / 10 ≈ 9,600 bytes, approximately 10 KB.

3. If we use a larger buffer, then CPU usage is reduced because the CPU needs to send new data less frequently (10 times a second for a 100ms buffer compared to 100 times a second for a 10ms buffer). The total copied size is the same every second, but the overhead around copying (more context switches…) is higher. However, latency is increased because the system might need to wait up to 100ms for new sound data to be sent to the sound card.

4. Without double buffering, we would need to wait for the end of the 100ms or 10ms period to send the new buffer to the sound card, causing a break in continuity of audio playback. Double buffering allows one buffer to be played while the other is being filled, maintaining continuous playback.

5. DMA improves efficiency by allowing the audio data to be transferred directly from memory to the audio controller without involving the CPU, freeing up CPU resources for other tasks and reducing the overhead associated with data transfer.